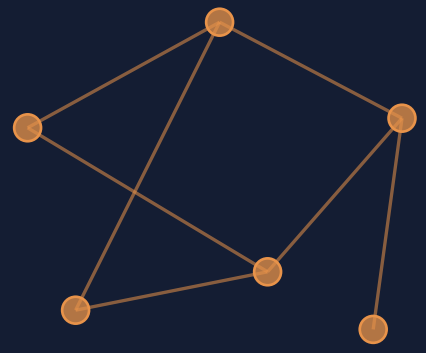
VERİ YAPILARI VE ALGORİTMALAR

1. Hafta — Giriş, Soyut Veri Tipi (ADT) ve Algoritma Analizi

Karadeniz Teknik Üniversitesi

Yazılım Geliştirme Bölümü

Dr. Öğr. Üyesi Sayyad Alizadeh



Dersin Yeri ve Ön Koşulları

- Bölüm müfredatında temel bir zorunlu ders niteliğindedir.
- **Ön koşul: C Programlama**
  - değişkenler, kontrol yapıları, fonksiyonlar, dizi ve işaretçiler
- **Uygulama dili: C (tüm örnekler ve ödevler)**
- Bu ders; Veritabanı Yönetim Sistemleri, İşletim Sistemleri ve İleri Algoritma derslerinin doğrudan temelidir.

Ön Koşul Zinciri



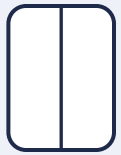
Haftalık Düzen: 3 saat teori + 1 saat uygulama (varsayılan — bölüm ders programına göre güncellenebilir)

Bu Haftanın Öğrenme Hedefleri

Bu haftanın sonunda öğrenciler...

1. Veri yapısı ile soyut veri tipi (ADT) arasındaki farkı açıklayabilir
2. Bir algoritmanın adım sayısından T(n) fonksiyonunu çıkarabilir
3. Big-O, Omega ve Theta notasyonlarını doğru yorumlayıp karşılaştırabilir
4. En iyi / en kötü / ortalama durum kavramlarını bir algoritma üzerinde ayırt edebilir
5. İkili arama algoritmasını adım adım uygulayıp karmaşıklığını hesaplayabilir
6. Derste kullanılacak pseudocode standardını okuyup C koduna çevirebilir

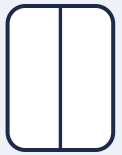
Ders Kaynakları



Fundamentals of Data Structures in C

Horowitz, Sahni, Anderson-Freed

- Konu sırası ve teorik derinlik için temel referans
- Karmaşıklık analizi ve asimptotik notasyon bu kitabın 1. bölümünü temel alır



Data Structures Using C

Reema Thareja

- C sözdizimi hatırlatmaları ve bol çözülmüş örnek için referans
- Uygulamalı alıştırmalar ve pekiştirme soruları için kullanılır

Her hafta için ilgili bölüm ve sayfa aralıkları haftalık sunumların son slaytında (Kaynakça) belirtilecektir.

Değerlendirme ve Ders İşleyişi

Bileşen	Ağırlık
Ara Sınav (Vize)	[%30]
Dönem Sonu Sınavı (Final)	[%70]
Uygulama Ödevleri	[%0]
Derse Katılım	[%0]

Bu tablo, bölüm yönetmeliğine ve her dönemde belirleneceği oranlara göre doldurulmaktadır.

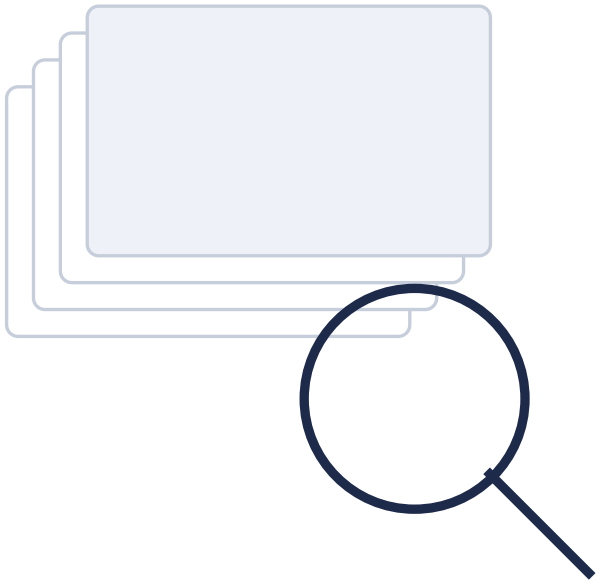
- Her hafta kısa bir kavrama sorusu / mini quiz ve haftalık ödev verilecektir.
- Uygulama ödevleri C dilinde derlenebilir kod olarak teslim edilmelidir.

Dönem Yol Haritası — 14 Hafta

1 Giriş & Analiz	**2** Diziler	**3** Bağlı Listeler	**4** Yığınlar	**5** Özyineleme	**6** Kuyruklar	**7** Ağaçlar I
8 Heap	**9** AVL / Splay	**10** B-Tree	**11** Graflar I	**12** Graflar II	**13** Sıralama	**14** Arama & Hashing

Bu hafta buradayız

Gerçek Dünya Problemi



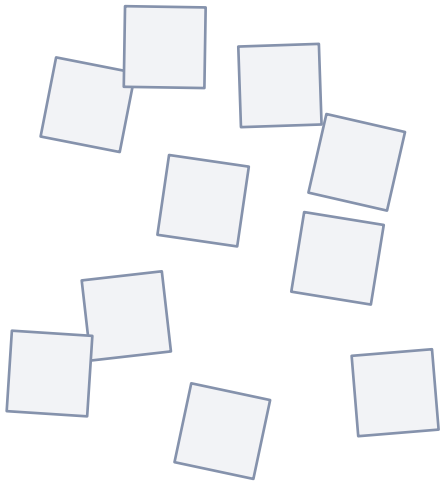
Senaryo:

50.000 öğrencilik bir üniversite otomasyon sisteminde, öğrenci numarası ‘20220001234’ olan öğrencinin kaydını bulmanız gerekiyor.

Soru:

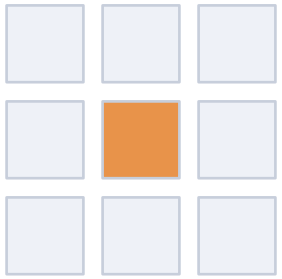
Bu kaydı bulmanın en hızlı yolu nedir? Kayıtların bellekte nasıl tutulduğu bu hızı nasıl etkiler?

Problemin Zorluğu



- Kayıtlar rastgele sırada tutuluyorsa, aranan değerin nerede olduğunu önceden bilemeyiz.
- Tek seçenek: baştan sona tek tek kontrol etmek (doğrusal arama).
- **En kötü durumda 50.000 karşılaştırma gerekebilir.**
- Veri büyüdükçe (N arttıkça) arama süresi de doğrusal olarak artar — 100.000 kayıtta bu sayı iki katına çıkar.

Çözüm: Organize Edilmiş Veri



Veriyi doğru biçimde organize edersek arama süresi dramatik şekilde azalır.

Bu dönem boyunca tam olarak bunu öğreneceğiz: her problem için doğru veri yapısını seçmek.

Sırasız arama: **50.000 karşılaştırma (en kötü durum)**

Sıralı + İKİLİ ARAMA: **yalnızca en fazla 16 karşılaştırma!**

Veri, Bilgi ve Veri Yapısı

Veri (Data)

İşlenmemiş, ham semboller veya sayılar. Tek başına anlam taşımaz.

Bilgi (Information)

İşlenmiş, anlamlı hale getirilmiş veridir.

Veri Yapısı

Verinin bellekte tutulma biçimi ve üzerinde tanımlı işlemler bütünü.

Önemli ayrım: "Veri" neyi saklıyoruz sorusuna, "veri yapısı" ise onu nasıl saklıyoruz ve nasıl kullanıyoruz sorusuna cevap verir.

Veri Yapısı Neden Önemli? (1) Zaman

Sırasız Liste — ‘Mehmet’ aranıyor

- Zeynep
- Ahmet
- **Mehmet**
- Elif
- Can

En kötü / ortalama durumda listenin büyük bölümü taranmalı.

Sıralı Liste — ‘Mehmet’ aranıyor

- Ahmet
- Can
- Elif
- **Mehmet**
- Zeynep

Sıralıysa ikiye bölerek çok daha az karşılaştırmayla bulunabilir (bu haftanın son bölümü: ikili arama).

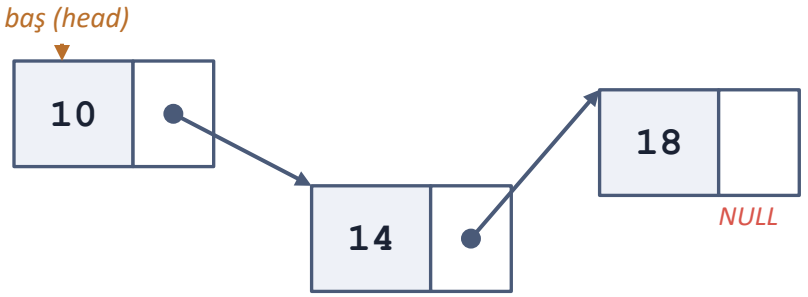
Veri Yapısı Neden Önemli? (2) Bellek Kullanımı

Diziler — Bitişik (ardışık) Bellek

10	14	18	22	26

Adresler ardışık → doğrudan hesaplanabilir, hızlı erişim. Ama araya eleman eklemek kaydırma gerektirir.

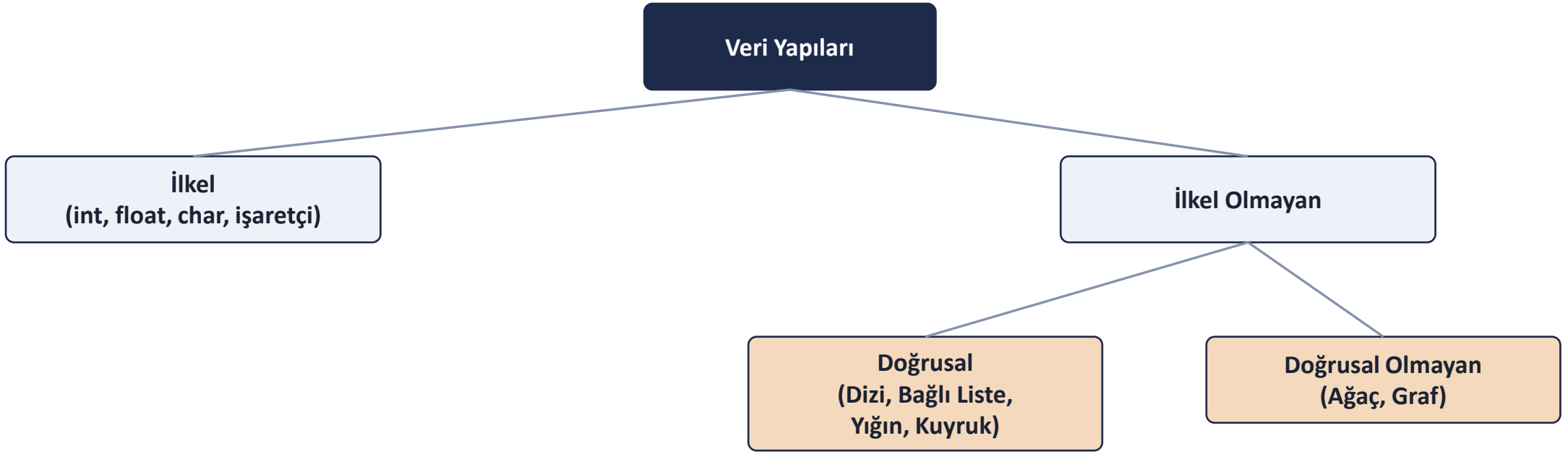
Bağlı Liste — Dağınık Bellek + İşaretçiler



Her düğüm; veri + bir sonraki düğümün adresini (işaretçi) tutar. Bitişik olması gerekmez → araya ekleme/silme ucuzdur, ama doğrudan indeksle erişim yoktur.

Bu ikisi arasındaki değiş-tokuş (trade-off), 3. haftada Bağlı Listeler konusunda ayrıntılı işlenecektir.

Veri Yapılarının Sınıflandırılması



Bu dönem ağırlıklı olarak 'İlkel Olmayan' dalındaki yapıları işleyeceğiz.

Bu Dönem İşleyeceğimiz Veri Yapıları



Dizi

2. Hafta



Bağlı Liste

3. Hafta



Yığın (Stack)

4. Hafta



Kuyruk (Queue)

6. Hafta



Ağaçlar

7–10. Hafta



Graflar

11–12. Hafta



Sıralama

13. Hafta

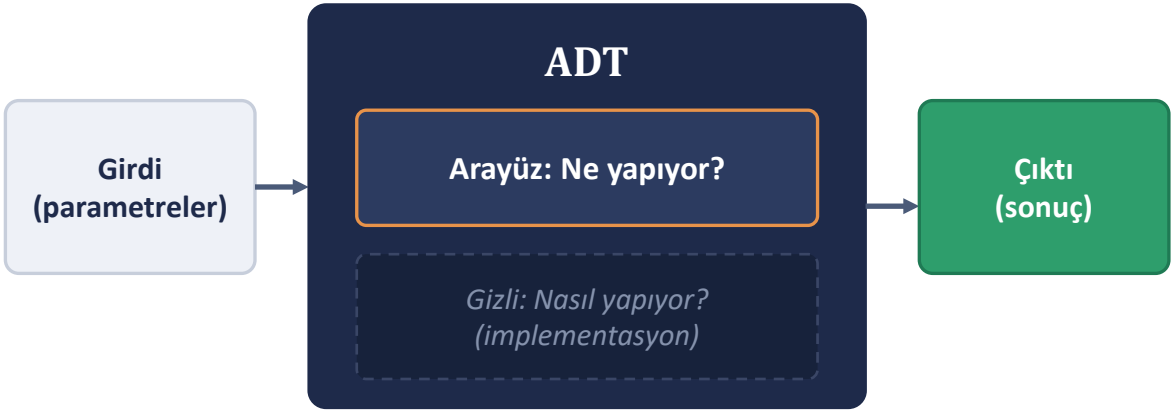


Hashing

14. Hafta

Soyut Veri Tipi (ADT) Nedir?

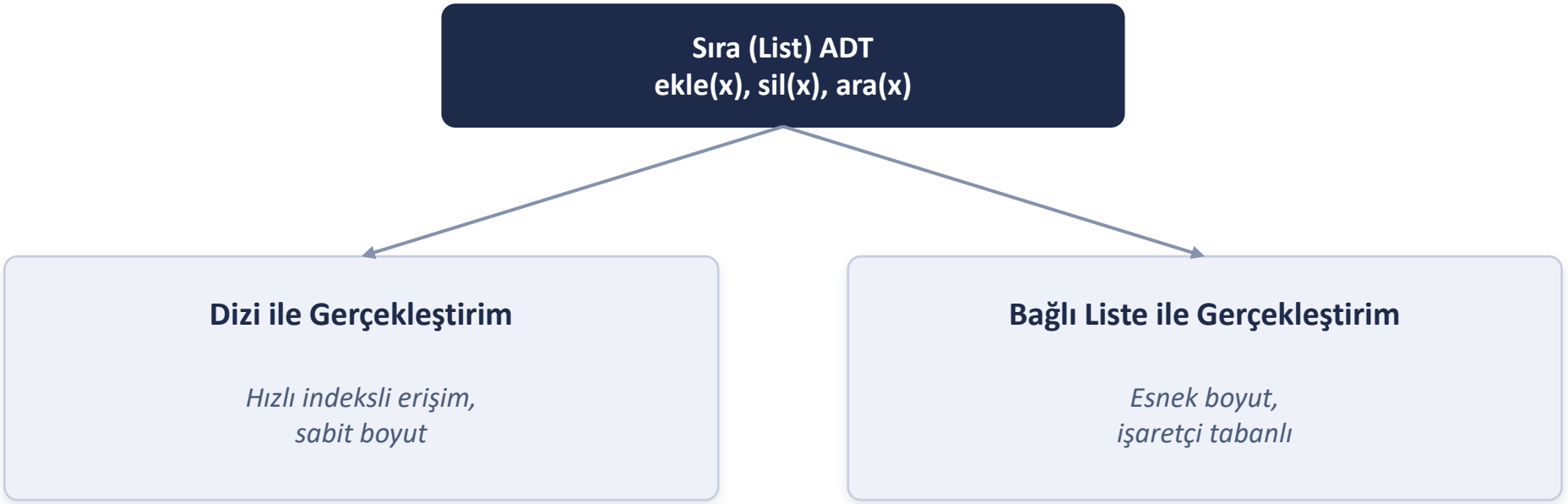
- ADT (Abstract Data Type); bir veri türünü yalnızca yapabildikleri (işlemler) üzerinden tanımlar.
- Nasıl gerçekleştirildiği (dizi mi, bağlı liste mi?) kullanıcıdan gizlenir.
- **Kısaca: "Ne yaptığı" bellidir, "nasıl yaptığı" bir uygulama detayıdır.**



Örnek: "Listeye eleman ekle" dediğinizde, bunun dizi içinde kaydırma ile mi yoksa yeni bir düğüm bağlayarak mı yapıldığını bilmenize gerek yoktur.

ADT Neden Önemli?

- Aynı ADT, farklı iç yapılarla gerçekleştirilebilir — kullanıcı arayüz değişmedikçe farkı hissetmez.



Kullanıcı için ikisi de aynı görünür — ekle(x) her ikisinde de çalışır, ama içerideki maliyetleri farklıdır (3. haftada ele alınacak).

Örnek ADT: Tam Sayı Kümesi

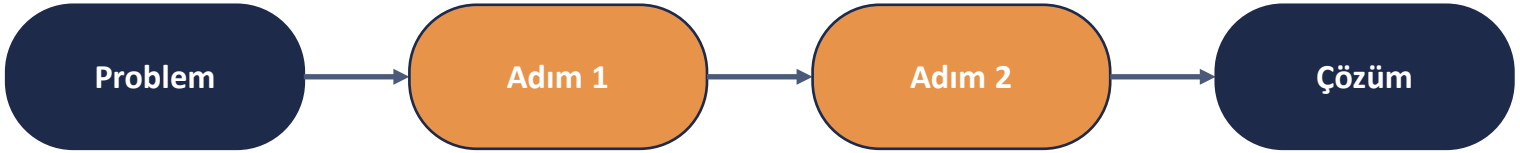
Kavram	Tanım
Nesneler (Objects)	Tekrarsız tam sayılardan oluşan bir koleksiyon
Ekle(S, x)	x elemanını S kümesine ekler
Sil(S, x)	x elemanını S kümesinden çıkarır
Üye_mi(S, x)	x, S içinde var mı? → doğru / yanlış
Boyut(S)	Kümedeki eleman sayısını döndürür

Dikkat: Bu tanımda kümenin bellekte nasıl tutulduğundan (dizi mi, ağaç mı?) hiç bahsedilmedi — sadece ne yaptığı tanımlandı.

Algoritma Nedir?

Bir problemi çözmek için izlenen, iyi tanımlanmış, sonlu adımlar dizisidir.

- Bir tarif (recipe) gibi düşünün: malzemeler (girdi), adımlar, sonuç (çıktı).
- Aynı algoritma farklı dillerde (C, Java, Python) yazılabilir — mantık aynı kalır.



Algoritmanın 5 Temel Özelliği

1

Girdi (Input)

Sıfır veya daha fazla dış veri alır.

2

Çıktı (Output)

En az bir sonuç üretir.

3

Kesinlik (Definiteness)

Her adım açık ve belirsizlikten arındırılmış olmalı.

4

Sonluluk (Finiteness)

Sonlu sayıda adımdan sonra mutlaka durmalı.

5

Etkinlik (Effectiveness)

Her adım; kalem-kağıtla dahi, sonlu sürede uygulanabilir olmalı.

Algoritma Tasarım Süreci



Doğrulama başarısız olursa 1. adıma (Problem Tanımlama) geri dönülür — bu süreç doğrusal değil, yinelemelidir (iterative).

Pseudocode Nedir, Neden Kullanılır?

Pseudocode; bir algoritmanın mantığını, herhangi bir programlama dilinin katı sözdizim kurallarına bağlı kalmadan, insanın okuyup anlayabileceği biçimde ifade etme yöntemidir.

- Dilden bağımsızdır — C, Java veya Python bilmeyen biri de anlayabilir.
- Sözdizimi hatalarıyla uğraşmadan, doğrudan çözüm mantığına odaklanılmasını sağlar.
- Herhangi bir programlama diline (bizim dersimizde: C) doğrudan çevrilebilir.
- **Bu derste her algoritma önce pseudocode, sonra C kodu olarak sunulacaktır.**

Bu Derste Kullanacağımız Pseudocode Standardı

Yapı	Gösterim	Örnek
Atama	`←`	`x ← 5`
Blok / kapsam	Girinti (indentation)	alt satırlar içeri kaydırılır
Koşul	`eğer / değilse`	`eğer x > 0 ...`
Döngü (sayılı)	`i ← 1'den n'e`	`i ← 1'den n'e adım 1`
Döngü (koşullu)	`iken (while)`	`iken i < n`
Fonksiyon	`Fonksiyon Ad(param)`	`Fonksiyon Ara(dizi, n, x)`
Döndür	`dön`	`dön bulunduMu`
Yorum	`// ...`	`// dizi taranıyor`

Pseudocode → C: Örnek (1/2)

Problem: Dizide en büyük elemanı bulma

Pseudocode

```
Fonksiyon EnBuyuk(dizi, n)
    enbuyuk ← dizi[0]
    i ← 1
    iken i < n
        eğer dizi[i] > enbuyuk
            enbuyuk ← dizi[i]
        i ← i + 1
    dön enbuyuk
```

Pseudocode → C: Örnek (2/2)

Aynı mantık, C sözdizimiyle

Pseudocode

```
Fonksiyon EnBuyuk(dizi, n)
  enbuyuk ← dizi[0]
  i ← 1
  iken i < n
    eğer dizi[i]>enbuyuk
      enbuyuk ← dizi[i]
    i ← i + 1
  dön enbuyuk
```

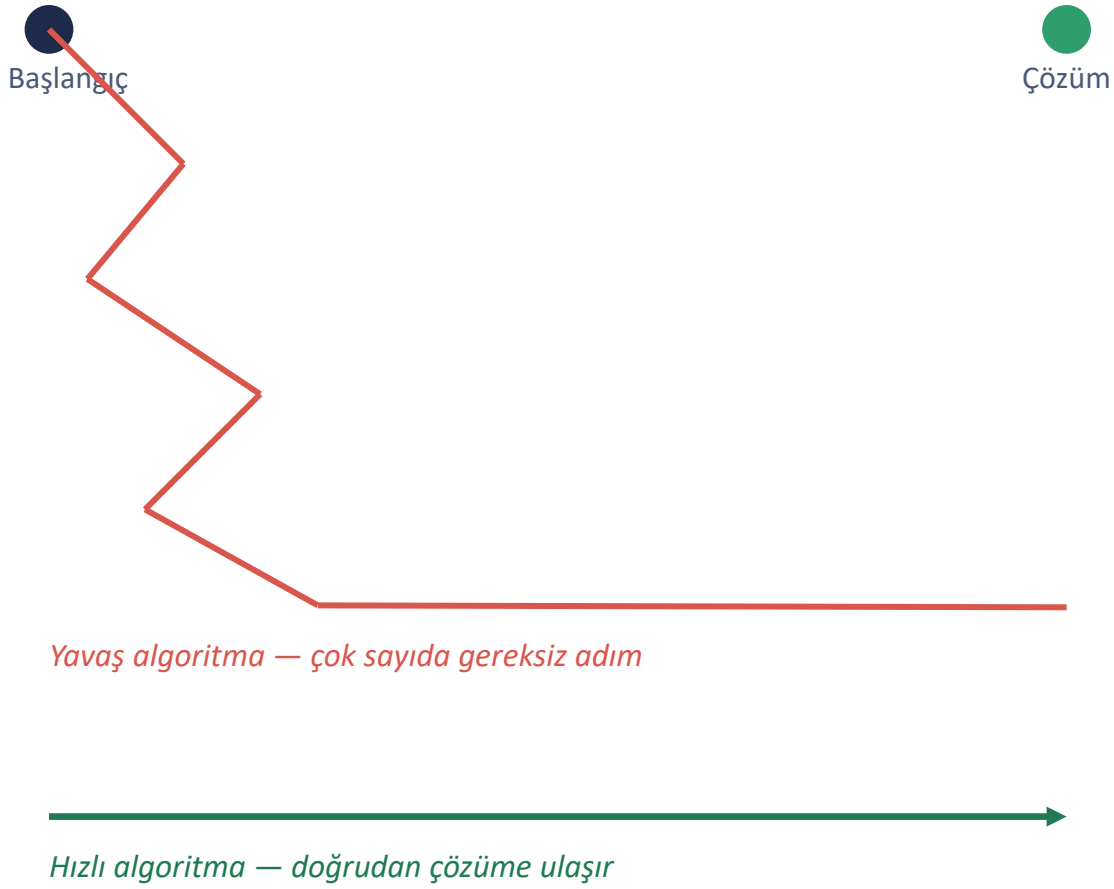
C Kodu

```
int enBuyuk(int dizi[], int n) {
    int enbuyuk = dizi[0];
    int i = 1;
    while (i < n) {
        if (dizi[i] > enbuyuk)
            enbuyuk = dizi[i];
        i = i + 1;
    }
    return enbuyuk;
}
```

Vurgulanan satırlar birbirine karşılık gelir: pseudocode'daki her adım, C'de bire bir bir ifadeye dönüşür.

Neden Algoritma Analizi Yapılır?

- Aynı problemi çözen birden fazla algoritma olabilir.
- Hangisinin daha ‘iyi’ olduğunu nasıl anlarız? Sezgi yeterli değildir, ölçü gerekir.
- **Algoritma analizi; bir algoritmanın kaynak (zaman / bellek) ihtiyacını matematiksel olarak tahmin etmemizi sağlar.**



Zaman Karmaşıklığı vs Alan Karmaşıklığı

Zaman Karmaşıklığı

Bir algoritmanın çalışması için gereken temel işlem (adım) sayısının, girdi büyüklüğü n'ye göre nasıl değiştiğidir.

Alan (Bellek) Karmaşıklığı

Bir algoritmanın çalışması için girdiye ek olarak ihtiyaç duyduğu bellek miktarıdır.

Değiş-tokuş (Trade-off): Bazen zamandan tasarruf için ekstra bellek harcanır (ör. önceden hesaplanan değerleri saklamak / memoization). Bu derste her algoritma için her ikisi de tartışılacaktır, ancak ağırlık zaman karmaşıklığına verilecektir.

Adım Sayma Yöntemi

- Her temel işlem (atama, karşılaştırma, aritmetik işlem) 1 adım olarak sayılır.
- Bir döngü n kez çalışıyorsa, içindeki adımlar n ile çarpılır.
- **Tüm adımlar toplanır → girdi büyüklüğü n'ye bağlı bir T(n) fonksiyonu elde edilir.**

Genel kural:

Ardışık ifadeler → adım sayıları toplanır

n kez çalışan döngü → içindeki adım sayısı n ile çarpılır

İç içe döngüler → adım sayıları birbiriyle çarpılır

T(n) Örnek 1: Dizi Toplamı

Kod satırı ↔ çalışma sayısı

```
int toplam(int dizi[], int n) {
    int s = 0;
    for (int i = 0; i < n; i++) {
        s = s + dizi[i];
    }
    return s;
}
```

Kod Satırı	Çalışma Sayısı
`int s = 0;`	1
`for kontrolü (i<n)`	1 + (n + 1) + n
`s = s + dizi[i];`	n
`return s;`	1

- *For döngü değişkeninin başlangıcı 1 kez çalışır.*
- *For döngüsünün kontrolü (i<n) n kez doğru, 1 kez de yanlış dönüp döngüyü bitirir → n+1 kez çalışır.*
- *For döngü değişkeninin artırma işlemi n kez çalışır.*

T(n) Örnek 1 — Sadeleştirme

```
T(n) = 1 + 1 + (n + 1) + n + n + 1
```

```
T(n) = 3n + 4
```

- Karmaşıklık analizinde sabitler ve daha düşük dereceli terimler önemsizleşir — n çok büyüdüğünde baskın terim belirleyicidir.

Sonuç: **T(n) = 3n + 4 → O(n)** *(Doğrusal karmaşıklık)*

T(n) Örnek 2: İç İçe Döngü

Kod satırı ↔ çalışma sayısı

```
int sayac = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        sayac++;
    }
}
```

Kod Satırı	Çalışma Sayısı
`dış for kontrolü`	n + 1
`iç for kontrolü (her dış adımda)`	n × (n + 1)
`sayac++; (her dış adımda n kez)`	n × n

Bu örneklerde T(n), tablolarda belirtilen seçilmiş işlemlerin toplamı olarak hesaplanmaktadır. Döngü değişkenlerinin başlangıç ve artırma işlemleri sayılmadığında katsayılar değişebilir; ancak asimptotik karmaşıklık sınıfı değişmez.

İç döngü, dış döngünün her bir adımında baştan sona çalışır — bu yüzden sayılar birbiriyle çarpılır, toplanmaz.

T(n) Örnek 2 — Sadeleştirme

$$T(n) = (n + 1) + n(n + 1) + n^2$$

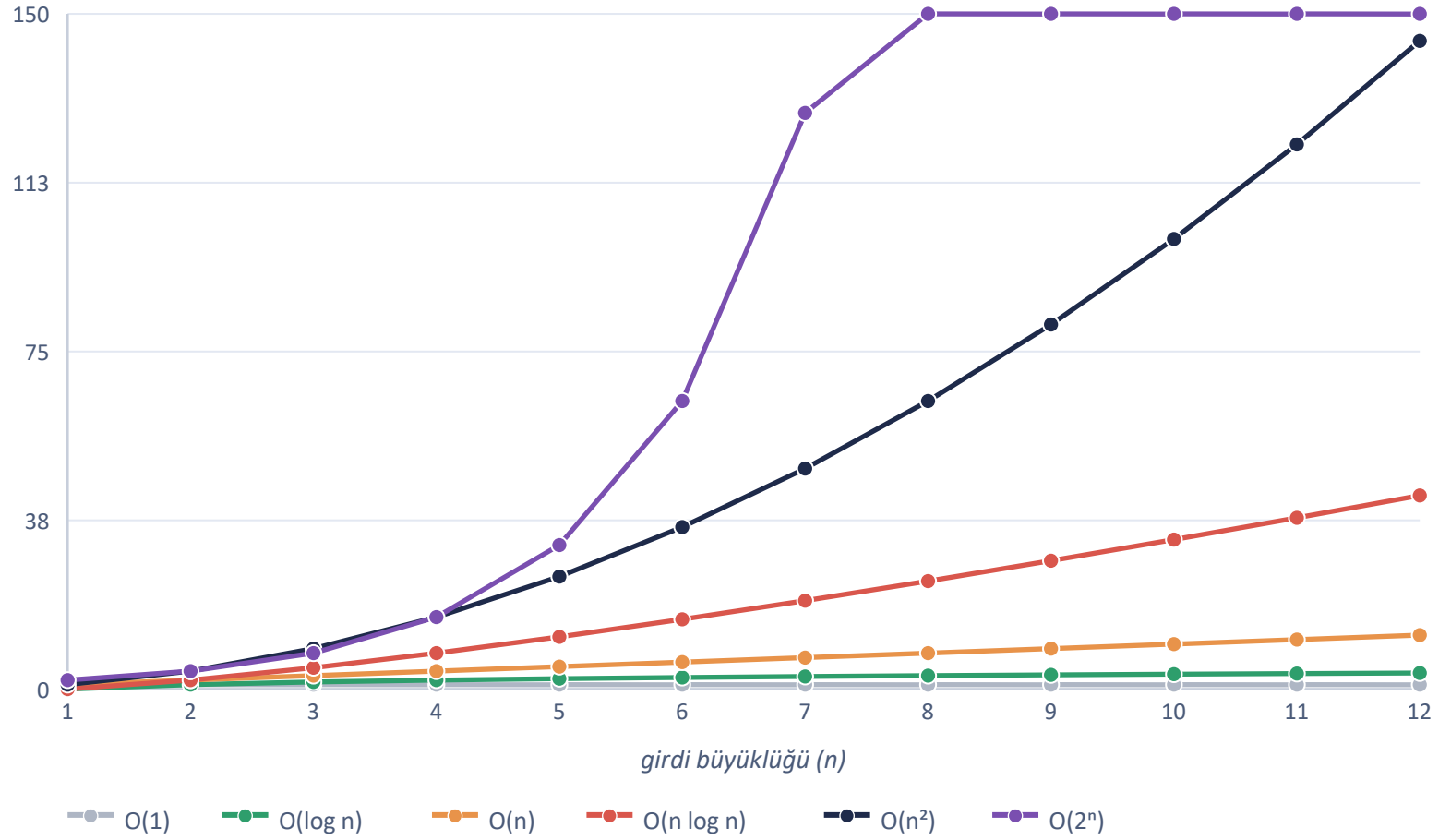
$$T(n) = n + 1 + n^2 + n + n^2$$

$$T(n) = 2n^2 + 2n + 1$$

- En yüksek dereceli terim (n^2) baskın olduğu için diğer terimler asimptotik sınıflamada elenir.

Sonuç: $T(n) = 2n^2 + 2n + 1 \rightarrow O(n^2)$ *(Karesel karmaşıklık)*

Karmaşıklık Sınıfları ve Büyüme Hızları



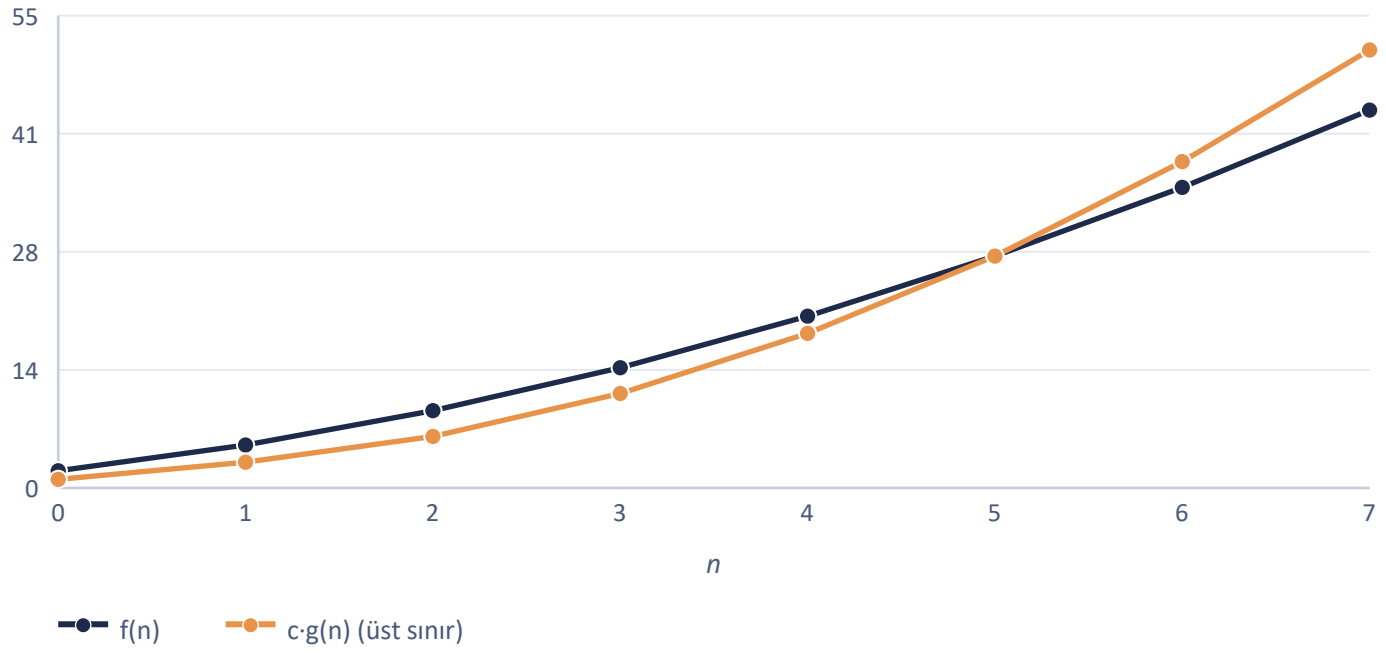
Dikkat:

O(2ⁿ) eğrisi n≈8'den itibaren grafik sınırını (150) aşar ve görünmez olur.

Bu kasıtlıdır: üstel büyüme o kadar hızlıdır ki küçük n değerlerinde bile grafik dışına çıkar.

Büyük-O (Big-O) Notasyonu: Tanım

`f(n) = O(g(n))`
eğer öyle pozitif c ve n_0 sabitleri varsa ki, tüm $n \geq n_0$ için $f(n) \leq c \cdot g(n)$ olsun.



Yorum:

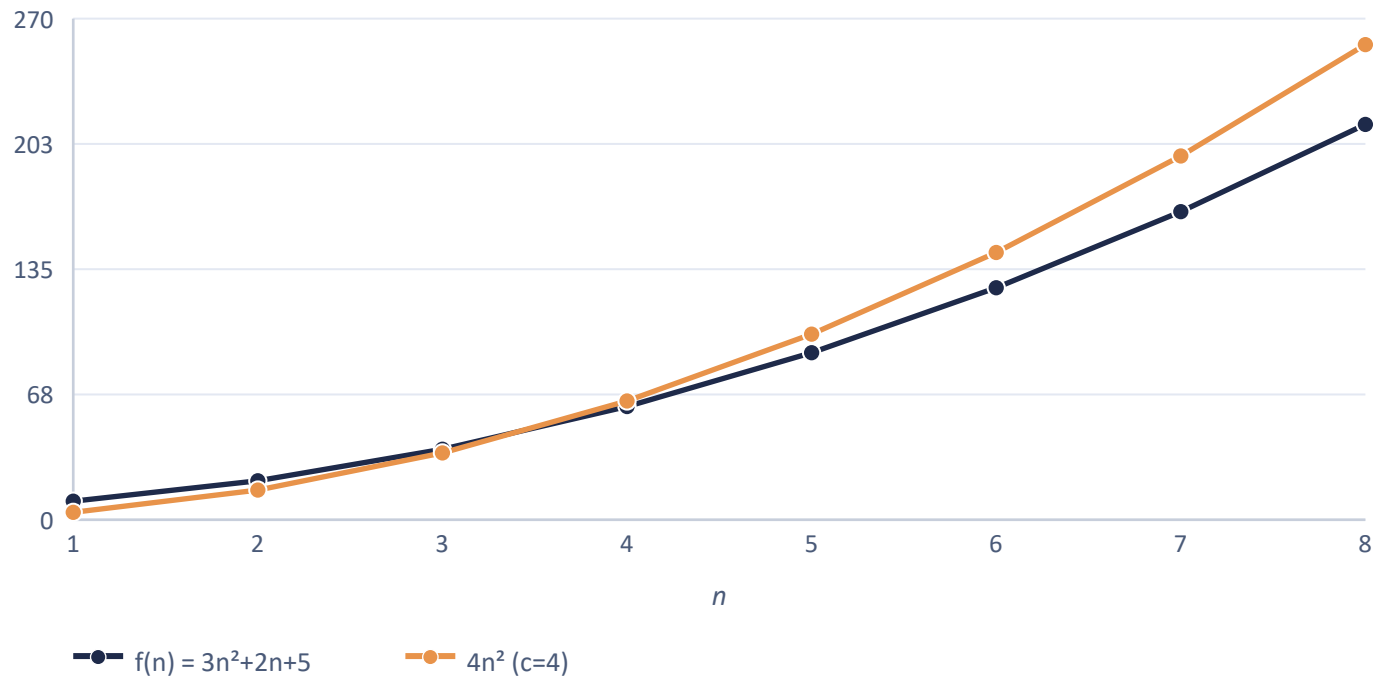
n_0'a kadar f(n) daha büyük olabilir.

$n \geq n_0$'dan itibaren c·g(n), f(n)'i artık hiç geçmemek üzere üstten sınırlar.

Big-O, bir fonksiyonun asimptotik üst sınırını ifade eder. En iyi, ortalama veya en kötü durum analizinde kullanılabilir.

Büyük-O Örnek 1

$f(n) = 3n^2 + 2n + 5$ — bu fonksiyon $O(n^2)$ midir?



İddia: c=4, n_o=4 seçersek,

n=4: f(4)=61, $4 \cdot n^2$=64 → 61≤64 ✓

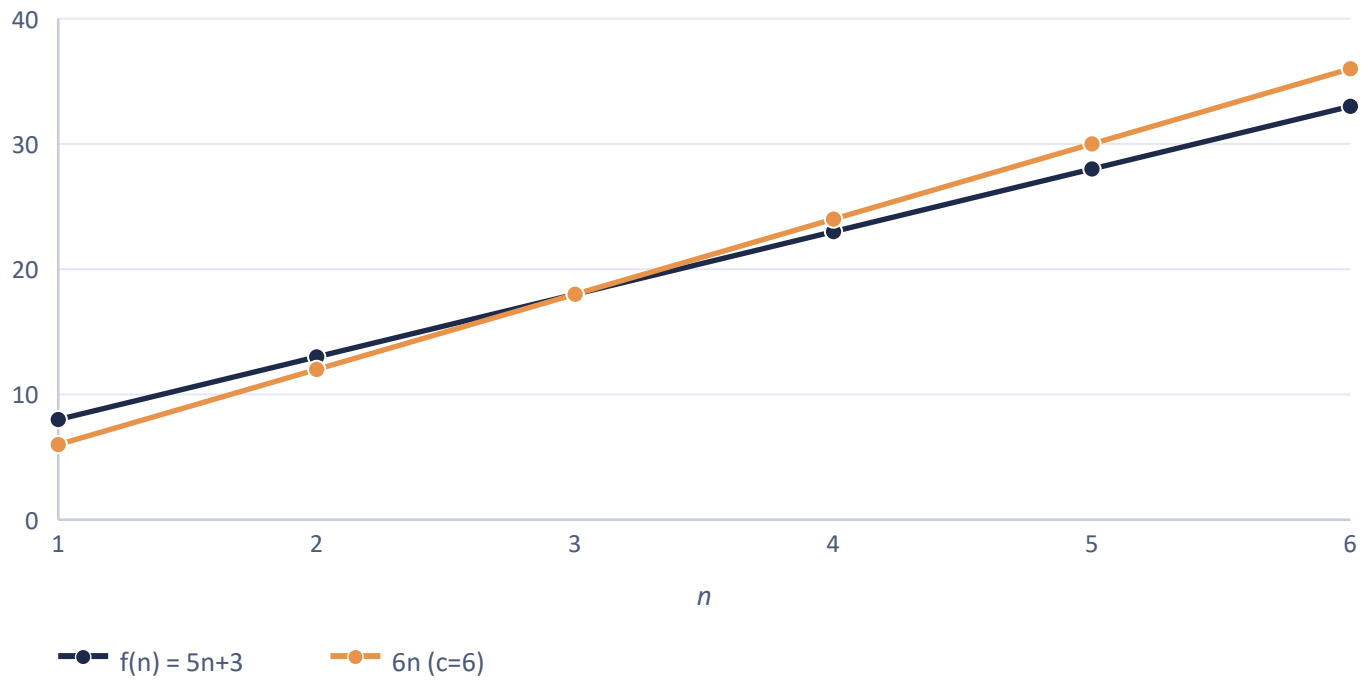
n=5: f(5)=90, $4 \cdot n^2$=100 → ✓

n=3: f(3)=38, $4 \cdot n^2$=36 → 38>36 ✗ (henüz sağlanmıyor)

Sonuç: $3n^2+2n+5 = O(n^2)$ *($c=4$, $n_o=4$ ile doğrulandı)*

Büyük-O Örnek 2

f(n) = 5n + 3 — farklı bir büyüme mertebesi



İddia: c=6, n_o=3 seçersek,

n=3: f(3)=18, 6·n=18 → 18≤18 ✓

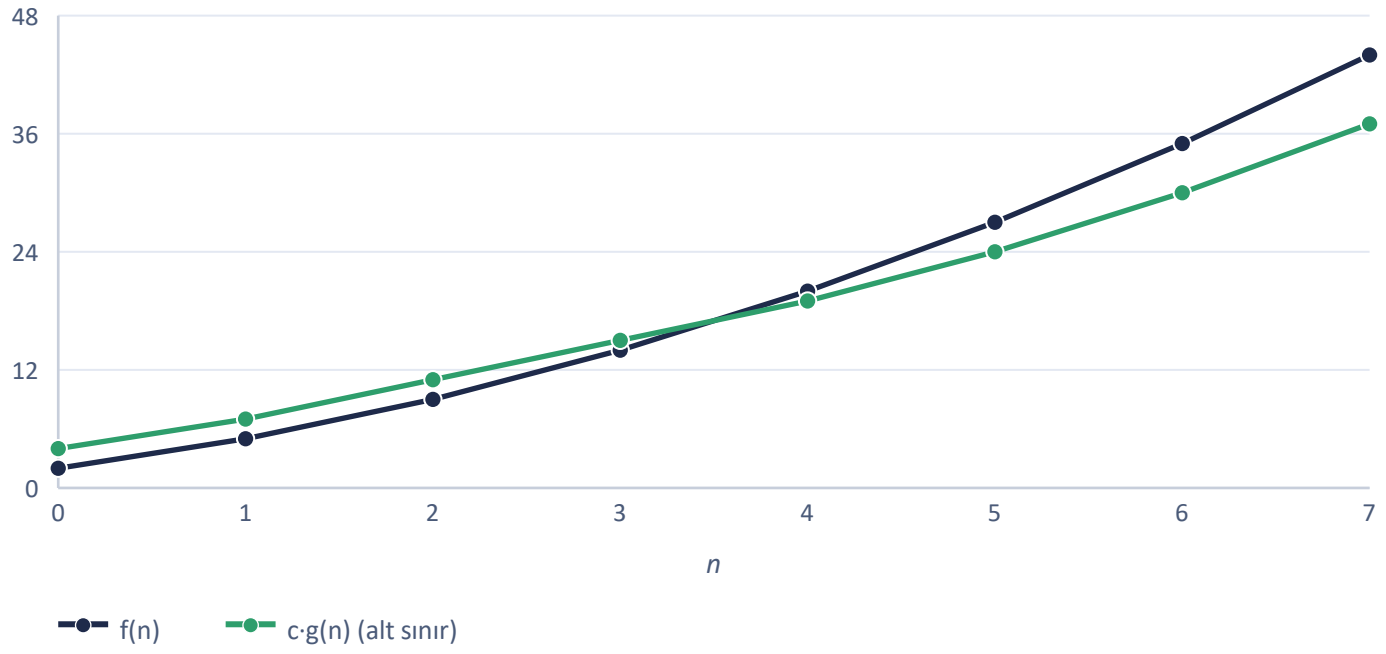
n=4: f(4)=23, 6·n=24 → ✓

n=2: f(2)=13, 6·n=12 → 13>12 ✗

Sonuç: 5n+3 = O(n) *(c=6, n_o=3 ile doğrulandı)*

Omega (Ω) Notasyonu: Tanım

f(n) = Ω(g(n))
eğer öyle pozitif c ve n_o sabitleri varsa ki, tüm $n \geq n_o$ için $f(n) \geq c \cdot g(n)$ olsun.



Yorum:

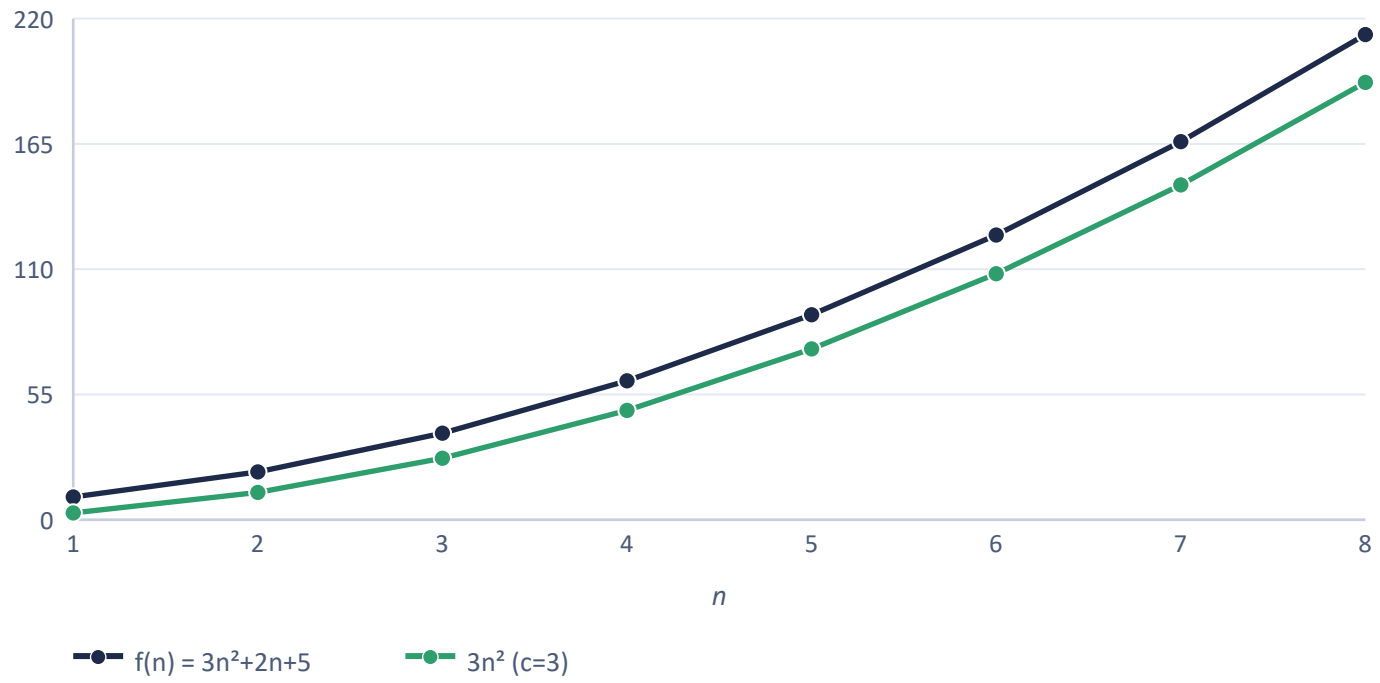
n_o'a kadar f(n) daha küçük olabilir.

$n \geq n_o$'dan itibaren c·g(n), f(n)'in artık hiç altına inmemek üzere alttan sınırı olur.

Omega (Ω), bir fonksiyonun asimptotik alt sınırını ifade eder. En iyi, ortalama veya en kötü durum analizinde kullanılabilir.

Omega Örnek

Aynı fonksiyon: $f(n) = 3n^2 + 2n + 5$



İddia: c=3, n_o=1 seçersek,

n=1: f(1)=10, $3 \cdot n^2$=3 → 10≥3 ✓

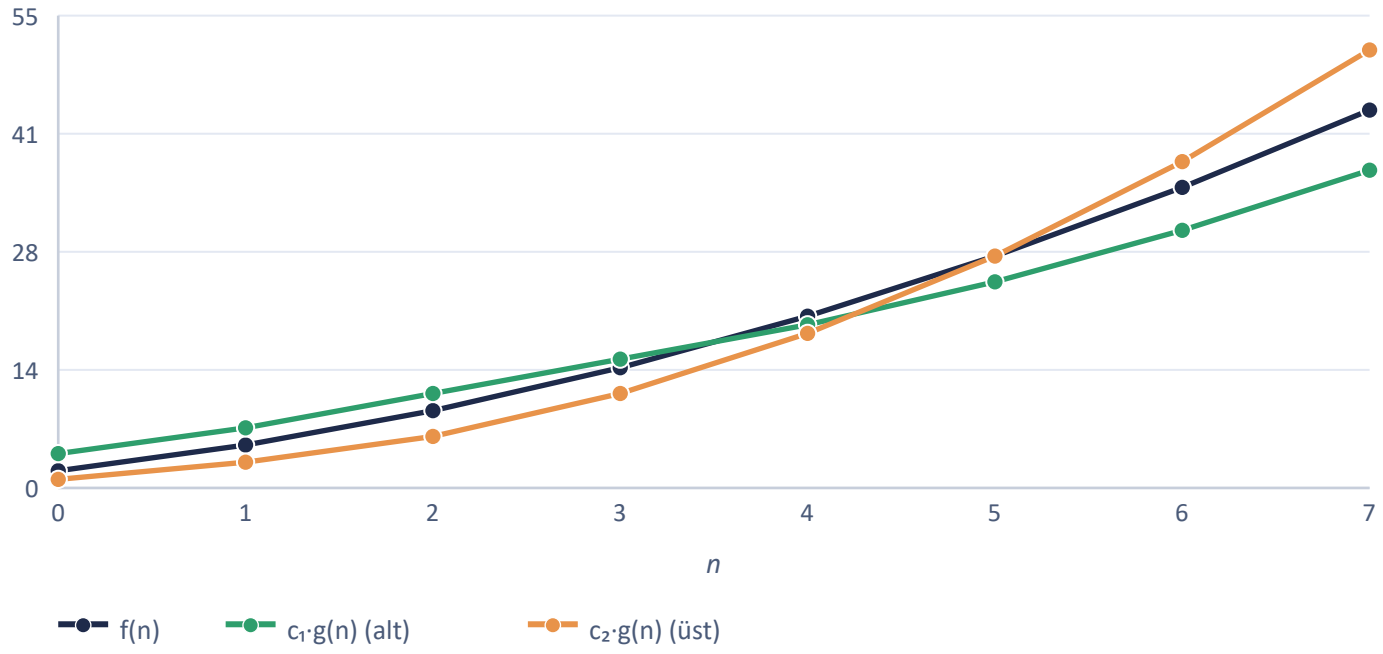
n=4: f(4)=61, $3 \cdot n^2$=48 → ✓

2n+5 > 0 olduğu için eşitsizlik HER n için baştan itibaren geçerlidir.

Sonuç: $3n^2+2n+5 = \Omega(n^2)$ *(c=3, n_o=1 ile doğrulandı)*

Theta (Θ) Notasyonu: Tanım

`f(n) = Θ(g(n))`
eğer öyle c_1, $c_2 > 0$ ve n_0 sabitleri varsa ki, tüm $n \geq n_0$ için $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ olsun.



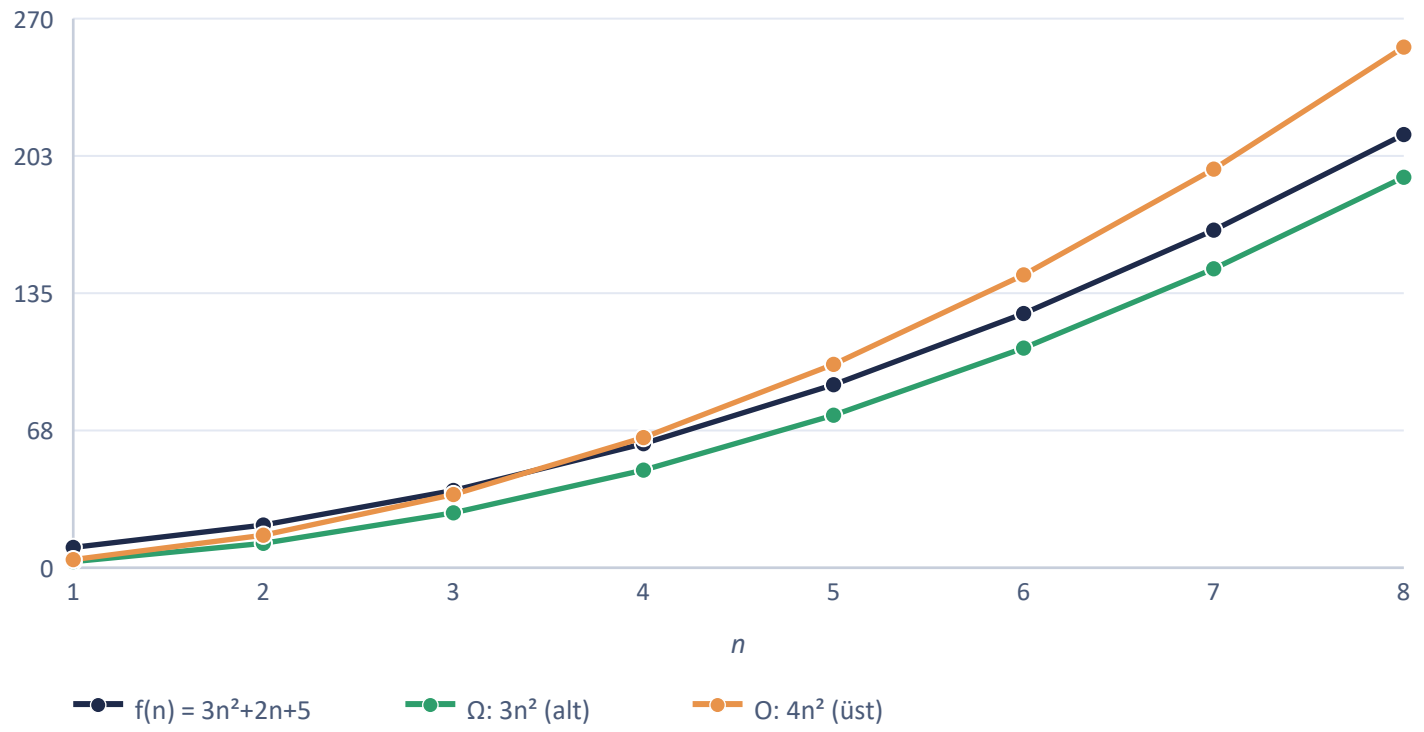
Yorum:

n_0'dan itibaren f(n), iki eğri arasına (bir "sandviçe") sıkışır.

Theta; büyüme hızının TAM olarak (sıkı) tanımıdır — hem alt hem üst sınır aynı anda geçerlidir.

O, Ω, Θ: Bir Arada Karşılaştırma

f(n) = 3n² + 2n + 5 — aynı fonksiyon, üç notasyon



n ≥ 4 (n_0) için:

$3n^2 \le f(n) \le 4n^2$

⇒ $f(n) = \Omega(n^2)$ (alt sınır)

⇒ $f(n) = O(n^2)$ (üst sınır)

⇒ $f(n) = \Theta(n^2)$ (sıkı sınır)

Bu, dersin geri kalanında kullandığımız üç notasyonun hepsinin aynı fonksiyon üzerinde nasıl bir arada çalıştığının özetidir.

En İyi / En Kötü / Ortalama Durum

En İyi Durum

`T_best(n)`

Algoritmanın en az adımda tamamlandığı senaryo.

En Kötü Durum

`T_worst(n)`

Algoritmanın en çok adım gerektirdiği senaryo. Big-O genellikle bunu ifade eder.

Ortalama Durum

`T_avg(n)`

Tüm olası girdiler üzerinden beklenen (ortalama) adım sayısı.

Bu derste bir sonraki bölümde her üç durumu da doğrusal arama üzerinde somut olarak göreceğiz.

Doğrusal Arama — En İyi Durum

hedef = 42



Senaryo:

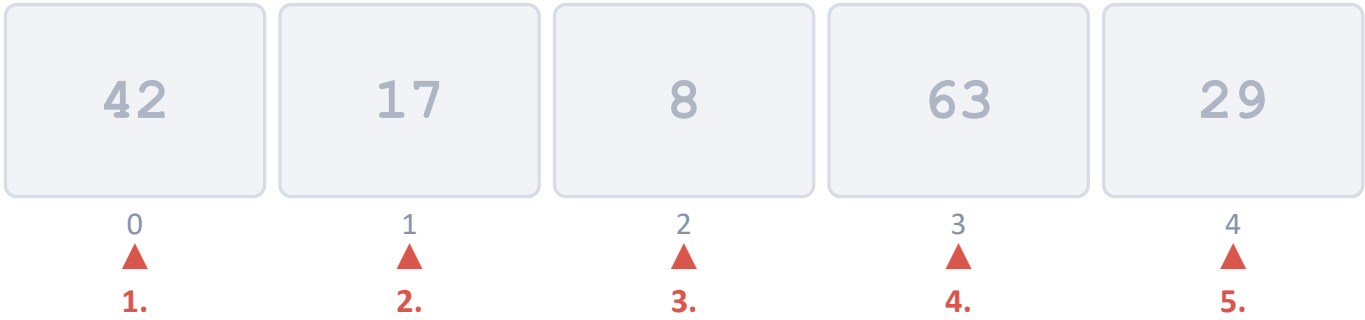
Aranan değer dizinin ilk elemanı.

Sadece 1 karşılaştırma yeterli.

T_best(n) = O(1) *— girdi büyüklüğünden bağımsız, sabit zaman*

Doğrusal Arama — En Kötü Durum

hedef = 99 (dizide yok)



Senaryo:

Aranan değer dizide yok (ya da son elemanda).

Tüm n eleman kontrol edilir.

T_worst(n) = O(n) — *Big-O genellikle bu durumu ifade eder*

Doğrusal Arama — Ortalama Durum

Hedefin eşit olasılıkla her pozisyonda olabileceğini varsayalım:




```
T_avg(n) = (1 + 2 + 3 + 4 + 5) / 5 = 15 / 5 = 3
Genel olarak: T_avg(n) = (1 + 2 + ... + n) / n = (n+1) / 2
```

T_avg(n) = (n+1)/2 → O(n) — *katsayılar elense de yine doğrusal*

Kapsamlı Uygulama: İkili Arama

Problem Tanımı



- Problem: Sıralı bir dizide belirli bir değerin (hedef) var olup olmadığını, varsa hangi indekste olduğunu bulmak.
- Yöntem: Dizinin ortasındaki eleman hedefle karşılaştırılır; hedef küçükse sol yarı, büyükse sağ yarı ile devam edilir. Arama aralığı her adımda yarıya iner.
- **Ön koşul: Dizi mutlaka sıralı olmalıdır.**

İkili Arama — Pseudocode

```
Fonksiyon IkiliArama(dizi, n, hedef)
    dusuk ← 0
    yuksek ← n - 1
    iken dusuk ≤ yuksek
        orta ← (dusuk + yuksek) / 2
        eğer dizi[orta] == hedef
            dön orta
        değilse eğer dizi[orta] < hedef
            dusuk ← orta + 1
        değilse
            yuksek ← orta - 1
    dön -1   // bulunamadı
```

İkili Arama — C Kodu

```c
int ikiliArama(int dizi[], int n, int hedef) {
    int dusuk = 0;
    int yuksek = n - 1;
    while (dusuk <= yuksek) {
        int orta = (dusuk + yuksek) / 2;
        if (dizi[orta] == hedef) {
            return orta;
        } else if (dizi[orta] < hedef) {
            dusuk = orta + 1;
        } else {
            yuksek = orta - 1;
        }
    }
    return -1;  // bulunamadi
}
```

İkili Arama — Adım 1 / 4

dusuk=0, yuksek=9 → orta=4



dizi[orta] = dizi[4] = 25 hedef = 41 → 25 < 41
⇒ Hedef daha büyük: SAĞ yarıda ara (dusuk ← orta + 1 = 5)

Kodda karşılık gelen dal

```
orta = (dusuk + yuksek) / 2;
if (dizi[orta] == hedef)       return orta;
else if (dizi[orta] < hedef)   dusuk = orta + 1;
else                           yuksek = orta - 1;
```

İkili Arama — Adım 2 / 4

dusuk=5, yuksek=9 → orta=7



dizi[orta] = dizi[7] = 55 hedef = 41 → 55 > 41
⇒ Hedef daha küçük: SOL yarıda ara (yuksek ← orta − 1 = 6)

Kodda karşılık gelen dal

```
orta = (dusuk + yuksek) / 2;
if (dizi[orta] == hedef)       return orta;
else if (dizi[orta] < hedef)   dusuk = orta + 1;
else                           yuksek = orta - 1;
```

İkili Arama — Adım 3 / 4

dusuk=5, yuksek=6 → orta=5



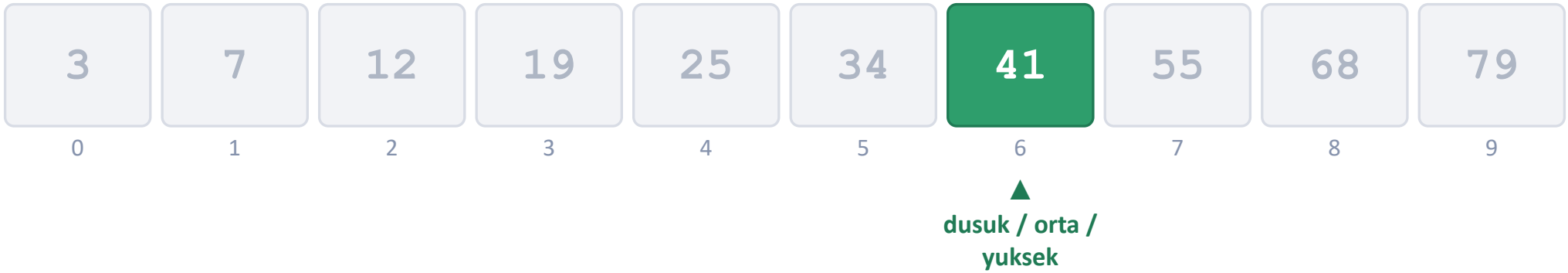
dizi[orta] = dizi[5] = 34 hedef = 41 → 34 < 41
⇒ Hedef daha büyük: SAĞ yarıda ara (dusuk ← orta + 1 = 6)

Kodda karşılık gelen dal

```
orta = (dusuk + yuksek) / 2;
if (dizi[orta] == hedef)       return orta;
else if (dizi[orta] < hedef)   dusuk = orta + 1;
else                           yuksek = orta - 1;
```

İkili Arama — Adım 4 / 4

dusuk=6, yuksek=6 → orta=6



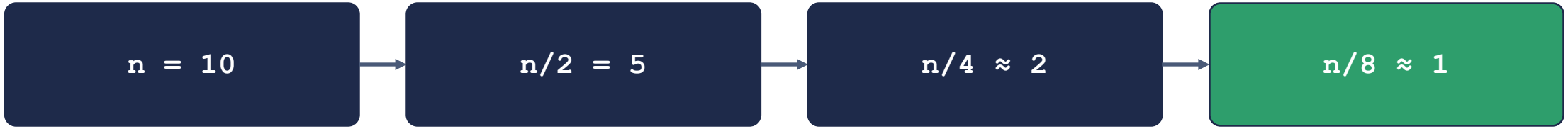
dizi[orta] = dizi[6] = 41 hedef = 41 → 41 == 41
✓ BULUNDU! dizi[6] = hedef — fonksiyon 6 değerini döndürür

Kodda karşılık gelen dal

```
orta = (dusuk + yuksek) / 2;
if (dizi[orta] == hedef)      return orta;
else if (dizi[orta] < hedef)  dusuk = orta + 1;
else                          yuksek = orta - 1;
```

İkili Arama — Karmaşıklık Analizi

- Her adımda arama aralığı YARIYA iner: n → n/2 → n/4 → n/8 → ... → 1



k adım sonra dizi boyutu: $n / 2^k$
Arama biterken $n / 2^k = 1 \Rightarrow k = \log_2(n)$

T(n) = O(log n) *— 10 elemanlı dizimizde $\log_2(10) \approx 3.32$, tavana yuvarlanınca 4 — tam da bulduğumuz adım sayısı!*

Genel Analiz Kuralları — Hafta Özeti

Toplama Kuralı

Ardışık kod bloklarının karmaşıklıkları toplanır: O(f) + O(g) = O(f+g)

Çarpma Kuralı

İç içe döngülerin karmaşıklıkları çarpılır: n kez çalışan bir O(f) işlem → O(n·f)

Baskın Terim Kuralı

Sadece en hızlı büyüyen terim kalır; sabitler ve düşük dereceli terimler atılır.

En Kötü Durum Varsayımı

Aksi belirtilmedikçe algoritmaların en kötü durum zaman karmaşıklığı Big-O ile ifade edilecektir.

Karmaşıklığın Pratikteki Önemi

Varsayım: bilgisayar saniyede 10^9 (1 milyar) işlem yapabiliyor

n	O(n)	$O(n^2)$	$O(2^n)$
10	10 ns	100 ns	≈ 1 µs
100	100 ns	10 µs	*evrenin yaşından uzun*
10.000	10 µs	0.1 s	*pratikte hesaplanamaz*

Önemli sonuç: n=100 gibi makül bir girdi büyüklüğünde bile, $O(2^n)$ karmaşıklığa sahip bir algoritma pratikte çalıştırılamaz. Doğru veri yapısı ve algoritma seçimi, bir problemi 'çözülebilir' ile 'çözülemez' arasında ayırabilir.

Haftalık Özet

- ✓ Veri yapısı ve ADT kavramlarını (arayüz vs implementasyon) ayırt ettik.
- ✓ Algoritma özelliklerini ve pseudocode → C çeviri sürecini öğrendik.
- ✓ Adım sayma yöntemiyle T(n) çıkarmayı iki örnek üzerinde uyguladık.
- ✓ Big-O, Omega ve Theta notasyonlarını aynı fonksiyon üzerinde birlikte gördük.
- ✓ En iyi / kötü / ortalama durum ayrımını doğrusal aramada somutlaştırdık.
- ✓ İkili aramayı 4 adımda uygulayıp O(log n) sonucunu doğruladık.

Gelecek Hafta Önizleme

2. Hafta: Diziler ve Bellek Temelleri



- 1B dizilerde indeks ve bellek adresi hesaplama
- Dizi üzerinde ekleme / silme / gezme işlemleri
- Temel struct ve işaretçi (pointer) kullanımı
- *(Ek materyal: 2B diziler, seyrek matris, polinom temsili)*

Hazırlık: Bu haftaki Big-O/Ω/Θ kavramlarını tekrar edin — 2. haftada dizi işlemlerinin karmaşıklığını analiz ederken doğrudan kullanılacağız.

Kavrama Soruları / Mini Quiz

1) $T(n) = 5n^2 + 100n + 2$ ifadesinin Big-O cinsinden karşılığı nedir?

2) Bir algoritmanın $T_{best}(n)=O(1)$ olması, $T_{worst}(n)$'nin de $O(1)$ olduğu anlamına gelir mi? Neden?

3) $f(n) = \Theta(n^2)$ diyebilmemiz için hangi iki koşul aynı anda sağlanmalıdır?

4) 20 elemanlı sıralı bir dizide ikili arama en fazla kaç karşılaştırmada sonuçlanır?

Haftalık Ödev

- 1) Aşağıdaki kod parçası için adım sayma yöntemiyle T(n)'i çıkarın ve Big-O cinsinden sadeleştirin (3 farklı kod parçası verilecektir).
- 2) $f(n) = 2n^3 + 5n + 1$ fonksiyonu için uygun c ve n_o değerleri seçerek $f(n) = O(n^3)$ olduğunu ispatlayın.
- 3) [4, 9, 15, 23, 31, 42, 50, 61, 77, 85, 93] dizisinde hedef=77 için ikili aramayı bu haftaki slayt formatında (her adım ayrı) elle çözün.
- 4) İkili arama fonksiyonunu C dilinde yazıp en az 3 farklı hedef değeriyle (dizide var / yok / ilk eleman) test edin.

Teslim: Bir sonraki ders öncesi, C kaynak dosyası + elle çözülmüş adımları gösteren bir PDF olarak.

Kaynakça

Kaynak	İlgili Bölüm(ler)	Sayfa Aralığı
Horowitz, Sahni, Anderson-Freed — Fundamentals of Data Structures in C	Chapter 1: Basic Concepts (1.2, 1.4)	s. 4–38
Thareja — Data Structures Using C	Chapter 2: Introduction to Data Structures and Algorithms	s. 43–65

Not:

İkili arama örneği ve karmaşıklık analizi anlatımı özgün olarak hazırlanmıştır; kaynaklardaki ilgili bölümler kavramsal çerçeve için referans alınmıştır.